Joint Robot Scheduling with Intersection Reservations and Charging

EE6228 Course Project (Topics M02 + M04)

Abstract

Four robots operating on two fixed directed loops must complete pickup-and-delivery tasks while sharing one intersection and limited charging bays. Task order, resource queues and battery state are coupled because waiting also consumes energy. This project jointly determines robot assignment, visit order, intersection slots, charging-bay selection and charging amount in a finite integer scheduling model. Joint Greedy supplies an initial schedule, CP-SAT searches for improvements, and a rolling version replans using only released tasks. Returned action traces are checked for precedence, resource capacity and battery safety. In archived 40-task experiments over ten seeds, CP-SAT reduces the mean composite objective J by 53.2 % and makespan by 28.3 % relative to FCFS (FCFS revision 75afc09; CP-SAT revision c87fb9f; eight workers and a 120 s CP-SAT limit). For a separate 12-task instance with seed 6, CP-SAT reduces makespan by 30.4 % under one worker, a 30 s wall cap and a 0.5 deterministic budget. Short-budget batch runs find no improvement beyond Joint Greedy. Overall, these findings highlight the potential of integrated task, intersection and charging scheduling to improve performance in fixed-route multi-robot systems.

Keywords: multi-robot scheduling; charging scheduling; constraint programming; intersection reservation; rolling horizon; discrete-event simulation.

1 Introduction

1.1 Research Background

Multi-robot systems are widely used for transportation and material handling in automated manufacturing and warehouse environments. Their scheduling begins with task allocation: deciding which robot performs each task and how the robots coordinate their work. Athira et al. [1] review the resulting problem classes and how coordination and uncertainty are handled.

In automated guided vehicle (AGV) systems, allocation interacts with shared traffic resources. Li, Fan and Jia [2] combine dynamic task assignment with conflict-free path planning in an automated container terminal, where vehicles may not occupy the same part of the network at the same time. A schedule that ignores such conflicts can become infeasible once vehicles meet.

Batteries add a second coupling. When chargers are scarce, charging competes with transport for both vehicle time and charger capacity; Li et al. [3] therefore schedule AGV tasks jointly with charging under a limited number of chargers. Building on these lines of work, this project investigates the joint scheduling of task execution, intersection reservations and charging decisions in a fixed-route multi-robot system. Methods for the individual model components are discussed where those components are introduced (Sections 3 and 4).

1.2 System Overview and Problem Statement

This project considers a multi-robot pickup-and-delivery system with four robots operating on two fixed directed loops. Two robots serve each loop, and each transport task requires a robot to collect and deliver an item between two stations on the same loop. The two loops share a single intersection, which admits only one robot at a time. The robots also share a charging area with up to two charging bays, each serving one robot at a time.

The scheduling problem is to determine which robot performs each task, the task execution order, when robots traverse the shared intersection, and when and how much they charge. These decisions must satisfy task precedence, resource capacity and battery safety constraints. The objective is to minimise a composite measure of makespan, priority-weighted tardiness, resource waiting time and energy consumption.

1.3 Motivation and Scheduling Challenges

Although each robot stays on its assigned loop for transport, all four share intersection X and the charging area. Task assignment determines when a robot reaches its pickup station, intersection X and a charging bay. Queues at shared resources then delay completion and consume battery energy. Makespan, tardiness, waiting and energy are therefore linked by the same assignment and timing decisions.

For example, when a robot (say R1) receives its next task, its battery may appear sufficient to complete the pickup and delivery yet still be insufficient to reach a charging bay safely afterwards. The scheduler must then arrange charging first. To reach the charging area, the robot must reserve and traverse the shared intersection. It may then need to wait for an available charging bay, consuming additional energy while queueing. After charging, the robot must reserve the intersection again to return to its loop. These extra movements and waiting periods may delay subsequent tasks and cause them to miss their deadlines. Task scheduling, resource reservations and battery state must therefore be considered jointly.

1.4 Research Questions

Based on these couplings, this project investigates the following questions:

- **RQ1:** Under identical tasks, maps, initial state-of-charge (SOC) values and offline information, how does joint decision-making affect makespan, total tardiness, resource waiting, energy consumption and the normalised composite objective J compared with First-Come, First-Served (FCFS) and staged methods?
- **RQ2:** What is the performance cost of requiring robots to retain enough energy at the end of the horizon to reach the charging area safely? How do task load, the number of charging bays and the distribution of tasks between Loops A and B shift the system bottleneck?
- **RQ3:** When tasks become visible only after their release time, what scheduling performance can the rolling-horizon method achieve, and how much replanning and solution overhead does it incur?

1.5 Contributions and Report Organisation

This project provides a joint scheduling model for task assignment, robot visit order, intersection reservations, charging-bay selection, charging duration and SOC evolution. Offline CP-SAT and rolling optimisation use the same discrete-event execution engine and output validator as the heuristic baselines. Every run produces its task inputs, actions, resource reservations, SOC traces and solver records, so aggregate results can be traced to individual decisions; the archived batch retains per-run metrics and solver records. The main contribution is this integrated framework and its evaluation on a fixed two-loop system.

Section 2 defines the system, its inputs and its scope. Sections 3–5 present the model, the solution methods and the implementation checks. Sections 6–9 describe the experiments, analyse the results, discuss limitations and conclude. The appendices give the full model, the worked experiment's task list and schedule records, and key code excerpts.

2 Problem Definition

2.1 Network and Resource Configuration

The system contains two unidirectional loops with fixed connectivity (Figure 2.1). Loop A follows X → A1 → A2 → A3 → A4 → WA → X and Loop B follows X → B1 → B2 → B3 → B4 → WB → X. Robots R1

and R2 serve only Loop A, and R3 and R4 serve only Loop B. WA and WB are the waiting points in front of the shared intersection X. The charging branch is X → H → C1/C2 → H → X, where H is the charging buffer and queueing location and C1 and C2 are configurable charging bays.

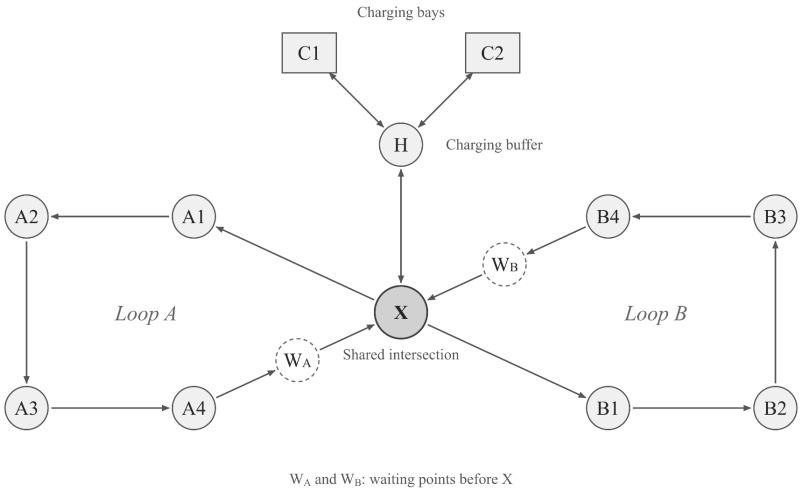


Figure 2.1. Network topology: two directed loops, the shared intersection X with waiting points WA and WB, and the charging branch through H to bays C1 and C2.

The map has 18 directed edges. Travel time equals edge length divided by speed, and movement energy equals edge length times a per-metre consumption rate; the map editor changes these parameters but never the connectivity. Only X and the charging bays are exclusive resources. Several robots may use an ordinary loop edge at once, so the model does not address geometric collision avoidance.

2.2 Tasks, Inputs and Outputs

Each transport task has eight fields: `task_id`; `release_time`, the earliest start; `pickup` and `dropoff`, two distinct stations on the same loop; `load_time` and `unload_time`; `due_time`; and an integer `priority` from 1 to 5. Table 2.1 shows sample records that illustrate this schema; Table B.1 gives the complete input for the 12-task experiment in Section 7.

Table 2.1. Sample input records from `examples/tasks.csv`.

task_id	release_time	pickup	dropoff	load_time	unload_time	due_time	priority
T036	30	A3	A1	20	20	787	1
T012	40	B4	B2	10	20	887	2
T016	40	B4	B3	15	15	825	3

System inputs comprise the task table, map parameters, initial robot SOC, safety threshold, bay count, charging rate, buffer parameters and solver options (CP-SAT time limit, workers and rolling horizon). Outputs comprise each task's assigned robot and completion time, per-robot action timelines, reservation intervals for X and the bays, SOC trajectories, validation results and summary metrics (makespan, weighted tardiness, waiting, energy, utilisation and J). JSON and CSV exports retain the complete trace for inspection and reproduction.

2.3 Scope and Assumptions

- **Fleet and network.** Four robots (two per loop), one shared intersection and up to two charging bays; each task is a single-loop pickup-and-delivery order, and a robot carries one task at a time. The experiments use 20–60 tasks.

- **Exclusive resources.** X and each enabled bay hold one robot at a time, with a safety buffer between consecutive crossings. Robots queue for X or a bay only at WA, WB or H and never stop inside X or queue inside an occupied bay.
- **Time and energy.** Durations are deterministic. SOC changes with movement, handling, queueing and linear charging; powered-down standby between tasks is free. Tasks, crossings and charging visits are non-preemptive, and a robot leaves its bay as soon as charging ends. The heuristics charge to 95 %, whereas CP-SAT chooses the charging amount.
- **Information.** The heuristics and offline CP-SAT see the whole task table but never start a task before its release; rolling CP-SAT sees only released tasks and keeps committed actions fixed. Manual command replay only executes and checks external plans and is excluded from the algorithm comparison.
- **Out of scope.** Nonlinear charging, multiple intersections, a variable fleet, continuous kinematics and obstacle avoidance. Robots need not return to an origin; the terminal reserve only keeps a charger reachable after the last task.

3 Mathematical Formulation

The model formalises the decisions within the scope of Section 2.3. Each task consists of empty travel to the pickup, loading, loaded travel and unloading.

3.1 Sets, Parameters and Decision Variables

Tables 3.1 and 3.2 define the notation. Times are in seconds; SOC and energy are in battery percentage points. Index 0 denotes the dummy initial vertex of each robot.

Table 3.1. Sets and parameters.

Notation	Meaning
$I,\ K,\ K_i$	Tasks, robots, and robots compatible with task i
$P,\ Q_k$	Enabled charging bays and candidate charging visits of robot k
$r_i,\ d_i,\ p_i$	Release time, due time and priority of task i
$L_i,\ U_i,\ g_h$	Loading and unloading times, and SOC consumption per handling operation
$\tau_{uv},\ g_{uv}$	Travel time and SOC consumption on directed edge (u, v)
$\rho_w,\ \rho_c$	Queueing drain and charging rate (SOC points per second)
$S_{\min},\ \beta,\ \delta$	Safety SOC floor, buffer parameter and crossing clearance $\delta = 2\beta$
$\hat{R}_C,\ \hat{R}_T,\ \hat{R}_W,\ \hat{R}_E$	Normalisation references for makespan, weighted tardiness, waiting and energy

Table 3.2. Decision variables.

Group	Notation	Meaning
Assignment and order	$x_{ik},\ a_{kab}$	Task assignment; arc selecting visit b after a for robot k
Task and visit times	$s_i,\ c_i,\ s_{kn},\ f_{kn}$	Task start and delivery completion; start and finish of visit n of robot k
Tardiness and makespan	$T_i,\ C_{\max}$	Task tardiness and the last delivery completion
Crossing reservations	$e_{kr},\ l_{kr},\ w_{kr}$	Crossing entry, exit and preceding queue
Charging decisions	$y_{kq},\ b_{kqp},\ u_{kq},\ D_{kq}$	Charging visit activation, bay selection, start time and duration
Battery state	$S^H,\ S^-,\ S^+$	SOC at H, before charging and after charging
Waiting and energy	$w^C,\ W_X,\ W_C,\ E$	Charging queue; total crossing and charging queues; positive SOC consumption

3.2 Assignment and Temporal Constraints

Each robot orders its assigned tasks and optional charging visits with `AddCircuit` through the dummy initial vertex. A task vertex is active exactly when the task is assigned; a charging vertex is active when the visit is enabled. Absent vertices use self-loops, and a dummy self-loop permits an unused robot. Assignment, arc, activation and bay variables are binary. Selected arcs impose

$$\sum_{k \in K_i} x_{ik} = 1, \qquad s_i \geq r_i, \qquad a_{kab} = 1 \;\Rightarrow\; s_{kb} \geq f_{ka} \quad (b \neq 0). \tag{3.1}$$

Here s_i is the task start (including empty travel to the pickup), c_i is the unloading completion and f_{ka} is the finish time of the preceding visit. For an assigned robot k, c_i satisfies Eq. (3.2): the selected predecessor determines the empty-travel time, loaded travel follows the fixed pickup-to-delivery route, and the sum adds the permitted resource queues over the set $\mathcal{R}_{ki}$ of crossings on the selected empty and loaded paths. The closing dummy arc ends the sequence without return travel, so the makespan equals the final delivery time.

$$c_i = s_i + \tau_{ki}^{\text{empty}} + L_i + \tau_i^{\text{loaded}} + U_i + \sum_{r \in \mathcal{R}_{ki}} w_{kr}. \tag{3.2}$$

3.3 Resource Capacity and Battery Safety

Every selected crossing reserves the interval $[e_{kr},\, l_{kr} + \delta)$ in one `NoOverlap` constraint, where e_{kr} and l_{kr} are the entry and exit times and $\delta = 2\beta$; the default buffer parameter of 2 s therefore gives a 4 s clearance between crossings. This models the intersection as a time-reserved resource, following reservation-based autonomous intersection management [4]. Robots wait at WA, WB or H and may not stop inside X.

Each selected charging visit queues at H, enters one chosen bay, charges without interruption and immediately returns to H; bay intervals are also non-overlapping. Bay selection satisfies $\sum_p b_{kqp} = y_{kq}$. With arrival time h at H, charging begins at $u = h + w^C + \tau_{Hp}$ and the visit ends at $u + D + \tau_{pH}$. Charging amount and duration are decision variables, as in AGV scheduling with flexible charging under a limited number of chargers [3] and variable charging durations [5]; the heuristics instead use a fixed 95 % target.

SOC decreases with movement, handling and resource queueing, while powered-down standby between tasks is free. For a segment without replenishment and with n_h handling operations,

$$S_{\text{out}} = S_{\text{in}} - \sum_{(u,v)} g_{uv} - n_h\, g_h - \rho_w \sum w, \tag{3.3}$$

which is checked before replenishment and after each action, within $[S_{\min}, 100]$. For a charging visit with SOC S^H on arrival at H, queue w^C and charging duration D, safe arrival, replenishment and departure require

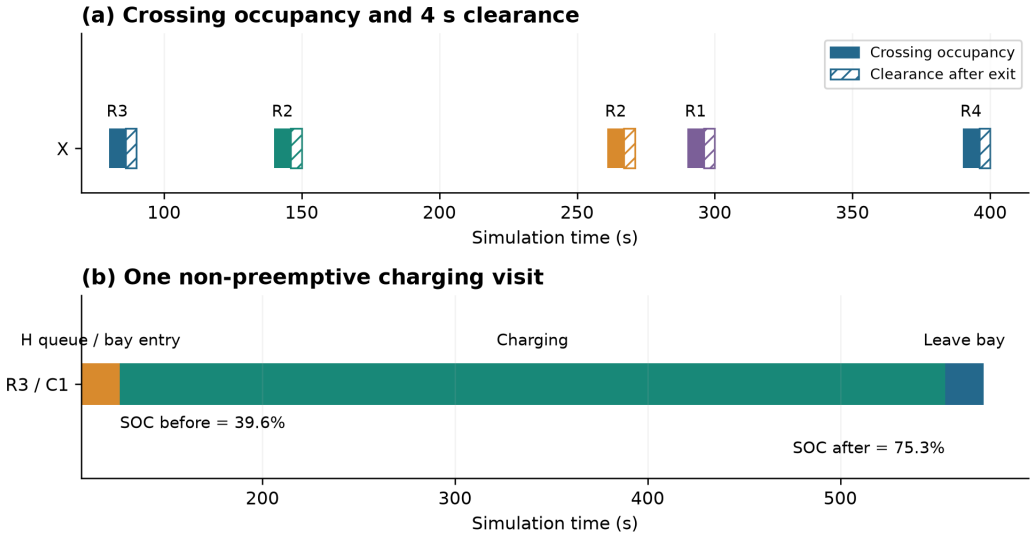
$$S^- = S^H - \rho_w w^C - g_{Hp} \geq S_{\min}, \qquad S^+ = S^- + \rho_c D \leq 100, \qquad S^+ - g_{pH} \geq S_{\min}. \tag{3.4}$$

The default terminal reserve requires every robot that serves tasks to keep enough energy after its final delivery to reach a bay and withstand conservative queueing, including an allowance for three other recharges; the integer model adds a 0.5-percentage-point margin, and the decoded plan is checked against the actual calendars. The `cpsat-nores` ablation removes only this terminal requirement and keeps battery safety during execution; it is evaluated separately because it permits more schedules. Rolling optimisation always keeps the reserve.

3.4 Objective and Model Scope

The optimiser minimises a composite objective of makespan, tardiness, waiting and energy. With $T_i = \max(0,\, c_i - d_i)$, $C_{\max} = \max_i c_i$, total intersection and charging queues W_X and W_C, and total positive consumption E,

$$\min\; J = \frac{1}{4}\left(\frac{C_{\max}}{\hat{R}_C} + \frac{\sum_i p_i T_i}{\hat{R}_T} + \frac{W_X + W_C}{\hat{R}_W} + \frac{E}{\hat{R}_E} \right). \tag{3.5}$$



Local 12-task case, seed 6, safety SOC 25%. Intervals are read from case_cpsat.json; hatching adds the modeled clearance.

Figure 3.1. Intersection reservations and a charging visit (local 12-task run, seed 6, safety SOC 25 %). Solid crossing intervals exclude the hatched 4 s clearance. The charging visit includes travel from H, charging and immediate departure.

Combining energy and delay follows multi-objective scheduling of multi-load AGVs that balances handling delays against energy consumption [6]. The references normally come from FCFS on the same instance, with denominator floors of 1 s, 60 s, 60 s and 1 SOC percentage point. If FCFS fails, Staged and Deadline are tried before floor-only references; with these fallbacks and floors, FCFS need not score exactly 1. Energy excludes replenishment, and weighted tardiness differs from the unweighted CSV field `total_tardiness`. Battery safety is a hard constraint, not a compensable objective term.

CP-SAT uses a 1 ms time grid, conservative integer approximations and a finite number of optional charging slots. OPTIMAL applies only to that bounded integer model; a time-limited feasible solution is not a physical global-optimality proof. All candidates must be decoded and validated, and a returned fallback does not inherit the solver's optimality certificate.

4 Solution Methodology

4.1 Baselines and Joint Greedy Scheduling

All automatic methods share the executor, map, energy rules and metrics. Table 4.1 summarises their decision rules. FCFS charges before a task when SOC is below 20 %; accepted tasks must also pass completion and terminal-reserve checks.

Joint Greedy considers up to eight pending tasks in release-time order and both compatible robots. It evaluates each candidate on a cloned state by attempting the task, checking its post-task reserve and inserting charging first if needed. Infeasible candidates are discarded without changing the committed schedule. The selected candidate minimises the local score

$$\text{score}_{ik} = c_i + 3\,p_i T_i + 0.5\,\Delta W_i + 2\,\Delta E_i, \tag{4.1}$$

with task and robot identifiers breaking ties; the incremental wait ΔW_i and positive consumption ΔE_i include any inserted charging visit. The score uses raw units and fixed coefficients rather than the normalised J. It gives a fast joint initial schedule, but its finite look-ahead does not guarantee the best complete schedule. Because the heuristics can inspect the full task list (while never starting a task before its release), they serve as offline comparators rather than baselines with the information restrictions of the rolling method. If no candidate passes the checks, the strategy reports that it found no safe plan; this does not prove that

Table 4.1. Baseline scheduling policies.

Policy	Decision rule	Comparison purpose
FCFS	Process by release time; choose the earliest predicted completion, including queues, charging and reserve checks	Simple executable reference
Staged	Fix assignment and order from nominal travel times, then add reservations and necessary charging	Cost of separate planning stages
Deadline	Order by due time, break ties by priority, check resource and battery feasibility	Due-date priority
Joint Greedy	Score up to eight pending tasks and compatible robots; commit the best feasible candidate	Joint heuristic and CP-SAT initial schedule

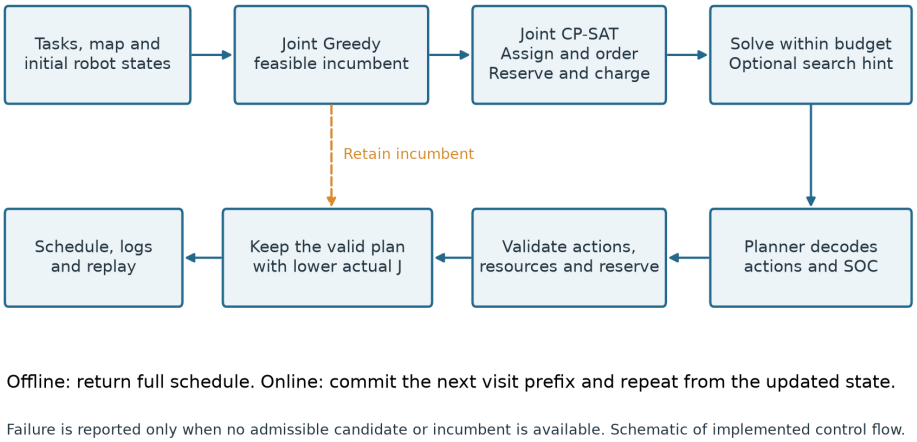


Figure 4.1. Joint planning and validation pipeline: solver, decoder, validator and incumbent fallback. The same execution rules apply to all automatic strategies.

no feasible schedule exists. Learning-based dispatchers, such as the hierarchical reinforcement-learning scheduler of Chang et al. [7], offer an alternative, but explicit rules are kept here so that every decision can be checked against the constraints.

4.2 Offline Joint Optimisation

Integrated machine–AGV scheduling has been formulated as a mixed-integer program [8] and as a constraint program that explicitly handles whether transport tasks exist [9]. This project likewise activates a visit only when it is selected, and solves the model with the CP-SAT solver of Google OR-Tools [10]. CP-SAT combines lazy clause generation with linear relaxations and a portfolio of parallel search workers, including large neighbourhood search [11]. It fits this problem because assignment and visit choices are Boolean, while optional intervals, `AddCircuit` and `NoOverlap` represent sequence and resource capacity directly; linear time and SOC constraints connect those choices to feasible execution. Joint Greedy supplies an initial physical schedule and, when it can be projected onto the grid, an `AddHint` search hint that guides the search without fixing decisions. The valid initial schedule is also retained outside the solver.

The simulator's planner decodes the selected circuits using the model's visit order, crossing times, bay choices and charging targets. It recomputes actual durations and SOC changes from the map instead of copying integer variables, and checks both the resulting actions and the enabled terminal reserve. The returned plan is the valid decoded candidate or the admissible heuristic incumbent, whichever has the lower actual J; this guarantees a comparison only with the retained Joint Greedy incumbent, not with

every baseline.

Solver status, bound, gap, decoded objective and fallback reason are stored separately. FEASIBLE means that the integer model has a candidate; a successful fallback is neither proven optimality nor a failed run. UNKNOWN means that the search has established neither feasibility nor infeasibility within its budget; INFEASIBLE concerns only the bounded integer model, and MODEL_INVALID is an implementation error. These statuses are kept separate from the validation result of any returned physical schedule. Overall runtime includes initial planning, model construction, solving, decoding and validation, whereas solver wall time covers search only. Under parallel, wall-clock-limited search, a fixed seed does not guarantee bitwise reproducibility; Section 6 gives the archived budgets.

4.3 Rolling Horizon Optimisation

At decision time t, rolling optimisation sees only released, uncommitted tasks. As in rolling-horizon allocation of dynamically arriving warehouse AGV tasks [12], the plan is recomputed as information arrives: by default every 60 s, over a window of roughly 600 s of workload capacity per robot. Tasks are prioritised by due time, priority and identifier. The window limits the work considered rather than imposing a completion deadline: travel to pickups, queueing or charging can extend the plan beyond it. Current robot states and committed reservations define the next solve's initial conditions. Receding-horizon re-optimisation has also been used to reorder AGVs through shared intersections during execution [13], much as the rolling variant here revises crossing and charging slots.

At each epoch, the planner commits only the prefix of each robot's visits that starts before the next decision epoch. The whole committed visit is frozen, including actions and reservations that extend beyond that epoch; ongoing tasks and charging are not interrupted, while uncommitted tasks may be reassigned later. Each solve normalises with the currently known tasks and state; full-instance references are used only for the retrospective score, so unreleased orders affect neither current decisions nor objective coefficients. Local solver bounds are not an online global-optimality certificate.

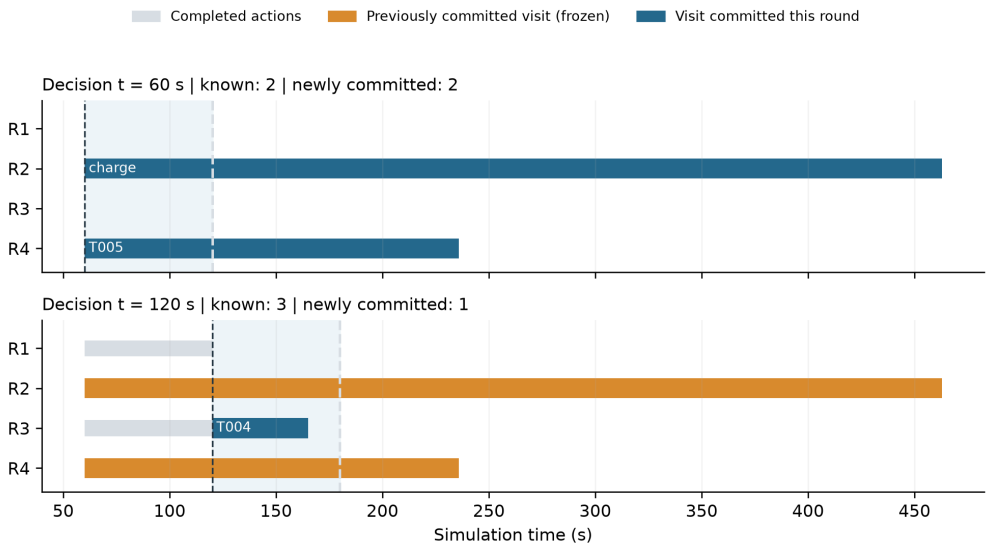


Figure 4.2. Rolling-horizon commitment in a local run, with decision epochs at 60 and 120 s. Visits committed in the first round remain frozen beyond the next decision epoch.

5 System Implementation and Validation

5.1 Shared Execution and Traceability

LoopLab combines a Python backend with browser replay. `simulator.py` maintains robot states, reservation calendars, action generation, metrics and validation; `optimizer.py` provides joint and rolling optimisation; `experiments.py` reuses instances and aggregates results. The front end visualises the returned trace rather than running another scheduler. Each action records robot and task identifiers, endpoints, start and end times, SOC, consumption and resources. Events, Gantt charts, battery curves and CSV/JSON exports all refer to the same run, so any aggregate result can be traced to specific actions and reservations.

Strict command replay provides a separate interface for `execute_task`, `move_to`, `charge` and `wait` commands with absolute start times. It rejects timing, resource and battery conflicts instead of queueing commands or inserting charging automatically; charging robots remain parked until explicitly moved, and omitted tasks remain unexecuted. Manual plans are therefore kept separate from the automatic algorithm ranking.

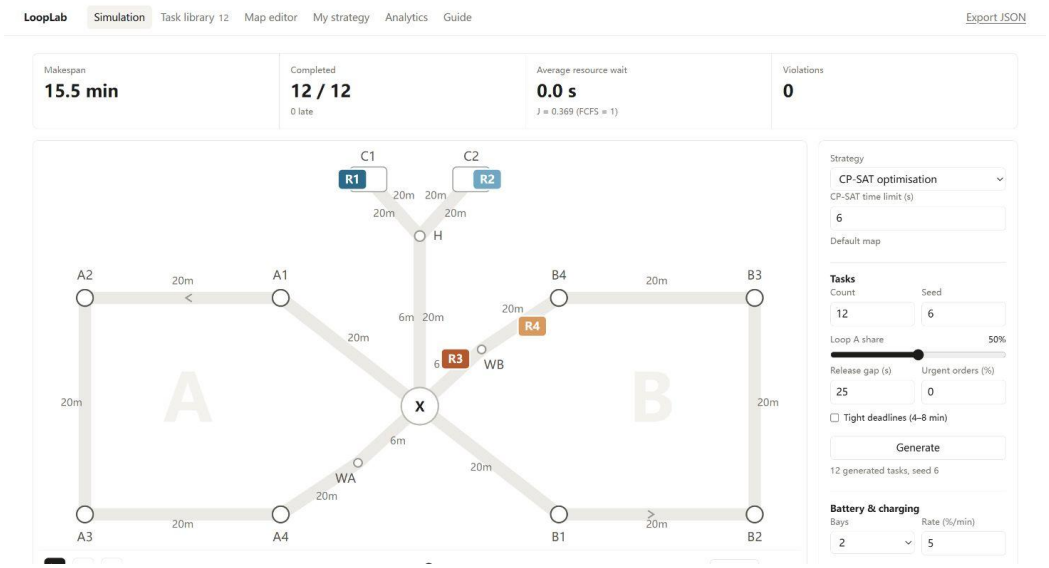


Figure 5.1. Local simulator replay: browser map, robot positions and summary metrics for a separate 12-task run. Its interactive solver settings differ from the controlled case in Section 7; quantitative comparisons use the saved case outputs.

5.2 Independent Validation

The validator recomputes correctness from the output tasks, actions, map and reservations. It checks unique task execution, release and pickup-before-delivery precedence, robot continuity and non-overlap, legal directed travel, load capacity, X and bay occupancy and clearance, legal waiting locations, and SOC continuity and bounds. Rebuilding occupancy from actions also detects missing or inconsistent reservation records. A candidate that fails these checks is rejected, even if the optimiser reports feasibility.

A one-task check uses A1 to A2, release 0, due time 600 s, priority 1, 15 s loading and unloading, and initial SOC 80 %. The default 20 m edge takes 20 s and consumes 2 SOC points, and each handling operation consumes 0.5, so completion is 50 s and final SOC is 77 %. With the FCFS references and floors, $J = \frac{1}{4}\left(\frac{50}{50} + \frac{0}{60} + \frac{0}{60} + \frac{3}{3}\right) = 0.5$. Table 5.1 compares this calculation with a fresh program run. A forged final SOC of 78 % fails the consistency check even though it exceeds the safety floor.

The objective is computed separately from the feasibility check: after validation, the result builder recomputes the last delivery time, task tardiness and priorities, resource queues and positive action consumption, and then applies the stored normalisation references. Repository tests check this calculation, and validator tests reject traces with altered timing, SOC or occupancy. The validator is independent

Table 5.1. Hand calculation and program output.

Quantity	Hand calculation	Program output
Completion time (s)	50.0	50.0
Energy (SOC points)	3.0	3.0
Final SOC (%)	77.0	77.0
Objective J	0.500	0.500
Constraint violations	0	0

of the planner's state but shares map and rule utilities, so its acceptance establishes conformity with the encoded model. The implementation and experimental data are available at `https://github.com/Eth4nMurpHy/robot-simulator` (code snapshot `9bf0b70`).

6 Experimental Setup

6.1 Evaluation Questions and Dataset Generation

RQ1 evaluates whether joint scheduling improves completion time, tardiness, waiting and energy relative to staged and single-rule baselines; RQ2 examines terminal reserves and resource sensitivity; RQ3 examines online solution quality and replanning cost. The experiments use synthetic tasks for the four-robot, dual-loop system.

For n base tasks, releases are sampled uniformly from a 10 s grid within $[0, 12n)$ seconds. Pickup and delivery stations are distinct and belong to the same loop. Loading and unloading each take 10, 15 or 20 s, and due times add 480–900 s to the release. Priorities 1, 2 and 3 have probabilities 50 %, 25 % and 25 %. Separate seeded streams generate urgent orders and per-robot initial SOC of 50–100 %, rounded to one decimal, so adding urgent orders preserves the base workload.

The synthetic inputs represent recurrent transport decisions rather than factory measurements. Same-loop orders respect the fixed guideway layout; release times create competing requests; handling durations and due allowances vary workload and urgency; and different initial SOC values create charging demand. The numerical ranges are experimental assumptions without calibration against operational records, so the results support controlled comparisons within this simulator.

6.2 Experimental Factors and Computing Environment

The base setting uses 40 tasks, two bays, a 10 % safety threshold, equal zone-assignment probabilities and no urgent orders. One factor varies at a time (Table 6.1). Each configuration uses seeds 1–10 and identical inputs across strategies. Ten distinct configurations × ten seeds × seven strategies yield 700 deduplicated strategy-run records; the CSV's 1 050 rows include repeated base results across experiment groups. Increasing the task count also lengthens the release window, rather than isolating arrival intensity within a fixed horizon.

Table 6.1. Experimental factors and solver budgets.

Factor	Levels or budget
Task count	20, 40, 60; release window $12n$ seconds
Charging bays	1 or 2
Zone imbalance	Expected A:B ratios 1:1, 2:1, 3:1
Safety SOC	10 %, 20 %, 30 %
Urgent orders	Additional 0 %, 10 %, 20 %; due allowance 180–360 s; priority 4 or 5
Solver budgets	Offline: 120 s, 8 workers. Rolling: 1 worker, deterministic budget 0.1 per solve, 30 s wall cap; 60 s replanning, approximately 600 s look-ahead

Archived experiments used Python 3.13 and OR-Tools 9.15.6755. Heuristics and rolling optimisation ran on two-vCPU Colab machines at commit 75afc09; offline CP-SAT used high-RAM Colab machines with Intel Xeon 2.20 GHz processors. The no-reserve runs used 75afc09, while reserve-preserving runs used c87fb9f with an SOC margin of 0.5 percentage points. Because these are historical results rather than a rerun of the current version, and hardware differs across methods, the recorded runtimes do not establish strict speed-up ratios. The rolling deterministic budget measures computational work, not 0.1 elapsed seconds.

The local illustration runs use snapshot 9bf0b70, Windows 11, Python 3.12.14 and OR-Tools 9.15.6755 on an AMD Ryzen 7 7840H; their seeds, budgets and sample sizes are reported with the results in Section 7. Historical and local observations are kept separate, and the one-task check in Table 5.1 uses the same snapshot.

6.3 Metrics, Statistics and Fair Comparison

Metrics include J, makespan, unweighted and priority-weighted tardiness, late-task rate, resource waiting, energy, charging sessions, runtime and violations. Average resource waiting is $(W_X + W_C)/|I|$, excluding powered-down standby, and energy sums positive SOC consumption without replenishment. Tables report means and sample standard deviations where shown; local figures show medians and individual seeds. The paired objective reduction is $1 - J_s/J_{\mathrm{FCFS}}$, so a positive value indicates improvement; it uses the actual FCFS J, which can be below 1 because of the denominator floors.

Heuristics and offline CP-SAT know future tasks but respect release times, whereas rolling CP-SAT sees only released tasks and normalises with known tasks until the final evaluation. The main comparisons keep terminal reserves; no-reserve CP-SAT is an ablation. Because heuristics charge to 95 % while CP-SAT selects charging amounts, the gains do not isolate charging effects. Feasible solutions do not establish optimality, and incumbent retention is not failure. All 700 archived records report success and zero violations; statistical significance has not been established.

7 Results and Analysis

The archive provides the ten-seed, seven-strategy comparison described in Section 6. Local figures use three seeds, four configurations and five offline strategies, giving 60 runs, plus a separate 12-task case with FCFS, CP-SAT and rolling plans. The archive supplies the broad comparison and the local traces explain individual decisions; because budgets and versions differ, the samples are not pooled.

7.1 Overall Scheduling Performance

Table 7.1 reports the archived base setting of 40 tasks, two bays and a 10 % safety SOC. Reserve-preserving CP-SAT achieves mean $J = 0.468$ against 1.000 for FCFS, a 53.2 % reduction. Mean makespan falls from 3 233 to 2 317 s (28.3 %), and consumption falls from 443.2 to 358.8 SOC points (19.0 %). These are separate outcomes: the reduction in J does not imply an equal reduction in completion time.

Table 7.1. Archived base performance over ten seeds. Tardiness, bay queueing and energy are means; tardiness is unweighted and energy excludes replenishment.

Method	J (mean ± SD)	Makespan (s)	Tardiness (s)	Bay queue (s)	Energy (SOC pp)
FCFS	1.000 ± 0.000	3233 ± 384	26 028	1 437	443.2
Staged	1.067 ± 0.156	3441 ± 543	27 264	1 487	449.0
Deadline	1.003 ± 0.240	3210 ± 499	25 994	1 290	438.3
Joint Greedy	0.793 ± 0.154	3021 ± 479	17 826	909	386.4
CP-SAT	0.468 ± 0.062	2317 ± 477	9 934	0	358.8
CP-SAT no reserve	0.349 ± 0.072	1732 ± 542	5 032	0	316.1
Rolling CP-SAT	0.636 ± 0.136	2741 ± 531	15 651	255	388.9

Joint Greedy reduces mean J by 20.7 %, whereas Staged and Deadline show little average advantage over FCFS: fixing assignment before reserving resources can leave charging conflicts for the execution stage to absorb. Rolling and no-reserve CP-SAT are shown separately because they use different information or constraints.

Interpreting J also requires a consistent reference. Each instance uses its own FCFS-based denominators, with lower bounds for small components, so FCFS can score below one, as in some 20-task local runs. A smaller J at one task count should not be read as proportionally less physical work than at another count; the paired comparisons, raw quantities and distributions together describe the outcome.

7.2 Solver Budget and Local Verification

Archived offline CP-SAT used 120 s and eight workers; its recorded solutions are FEASIBLE, so the results are not proofs of optimal schedules. In the base setting, total runtime averages 120.67 s for CP-SAT against 0.29 s for Joint Greedy. Because the methods ran on different Colab machines, these times do not support a strict speed-up comparison, though they illustrate the quality–computation trade-off under the recorded budgets.

The local check uses one CP-SAT worker, a deterministic budget of 0.15 and a 6 s wall cap per solve. Figure 7.1 shows every seed for the 40-task base setting. All 12 local CP-SAT runs return the same J as Joint Greedy, and nine explicitly keep the incumbent through fallback, so no improvement beyond the initial schedule is established at this budget. All 60 batch outputs pass validation, as do the three case plans and the browser-exported plan, and the repository test suite passes all 69 tests.

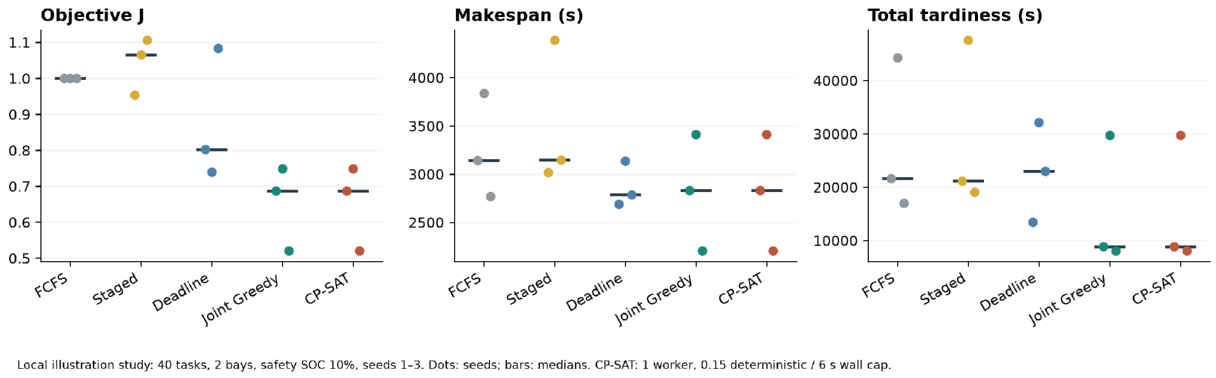


Figure 7.1. Local algorithm comparison: three seeds, 40 tasks, two bays, safety SOC 10 %. Dots show individual runs and horizontal marks show medians. CP-SAT and Joint Greedy coincide under the short budget.

The solver records explain the overlap in Figure 7.1. The nine fallback runs have status UNKNOWN because the search finds no CP-SAT incumbent within the budget; they still return a validated Joint Greedy plan. The other three runs, all with 20 tasks, are FEASIBLE with reported gaps of about 36–64 %, and their returned J still matches the initial plan. The local 12-task offline solution is also FEASIBLE, with a gap of about 56 %. None of these cases supplies an optimality certificate. Counting fallback as failure would hide the executable schedules; counting it as solver success would hide the search limitation.

7.3 Worked Example and Bottleneck Analysis

The local case has 12 tasks, seed 6, a release gap of 25 s, two bays and a 25 % safety floor. Initial SOC for R1–R4 is 50.1, 53.9, 52.2 and 80.8 %. Offline CP-SAT uses one worker, a deterministic budget of 0.5 and a 30 s wall cap; its measured total runtime is 2.62 s. Figure 7.2 uses identical time axes for FCFS and CP-SAT, and Table 7.2 includes rolling on the same input, which is listed in full in Table B.1.

Under FCFS, R1 waits at H from 116.0 to 750.4 s, i.e. 634.4 s of charging queue, and its recharge then lasts until 1 486.2 s. CP-SAT charges R3 first from 126.0 to 554.4 s and then admits R1 immediately at 554.4 s; R1 finishes charging at 868.8 s. The three CP-SAT charge targets are 75.3, 74.3 and 52.1 % rather than 95 %. These changes remove bay queueing and shorten the schedule: makespan falls by 30.4 % and

Table 7.2. Local 12-task case.

Method	J	Makespan (s)	Tardiness (s)	Bay queue (s)	Minimum SOC (%)
FCFS	1.000	1673.2	2518.4	634.4	35.4
CP-SAT	0.447	1165.2	275.2	0.0	25.9
Rolling CP-SAT	0.505	1237.6	1011.0	0.0	33.4

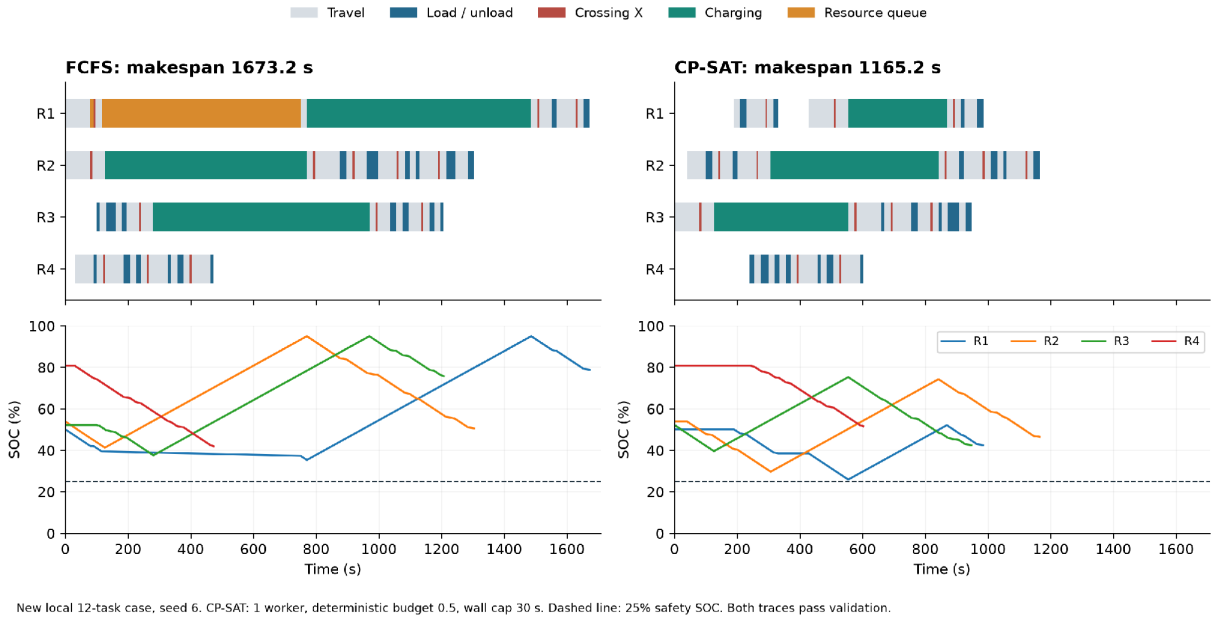


Figure 7.2. Local schedule and SOC comparison. The upper panels show robot actions and the lower panels SOC; the dashed line marks the 25 % safety floor. Both schedules keep terminal reserves and pass validation.

unweighted tardiness by 89.1 %. Energy remains almost unchanged (160.548 versus 160.400 SOC points), so the case supports a timing and charging explanation rather than an energy-saving claim. Zero queueing also does not mean zero idle time. Average resource waiting falls from 53.7 s per task to zero; this includes the 10 s of FCFS intersection queueing as well as its bay queue.

7.4 Terminal Reserve and Resource Sensitivity

In the archived base setting, dropping the terminal reserve lowers mean J from 0.468 to 0.349 and makespan from 2 317 to 1 732 s. Battery safety during execution remains enforced; only the post-batch reserve is removed. The lower values therefore describe a larger feasible region, not a better method under identical requirements, and for this ablation the summary files do not show that a charger stays reachable after the final task.

Figure 7.3 shows the local Joint Greedy median J rising from 0.449 at 20 tasks to 0.687 at 40 and 0.860 at 60. At 40 tasks, reducing capacity from two bays to one raises the median charging queue from 1 604.6 to 3 913.8 s for FCFS but only from 910.4 to 1 012.2 s for Joint Greedy, so scheduling changes how strongly a resource shortage is felt. Three seeds show these observed cases without establishing statistical significance.

The archived sensitivity results cover conditions beyond the local check. With the expected A:B task ratio changed from 1:1 to 3:1, the mean makespan of reserve-preserving CP-SAT increases from 2 317 to 3 010 s. Raising the safety floor from 10 % to 30 % increases its late-task fraction from 32.5 % to 48.0 %, and adding 20 % urgent tasks increases mean makespan to 2 820 s and the late fraction to 48.8 %. Constraints and workload can therefore limit delivery performance even when every returned plan is feasible. These factors answer different questions: bay count tests resource availability, zone imbalance tests where work is concentrated, and the safety SOC reduces the usable battery range. Urgent orders change the num-

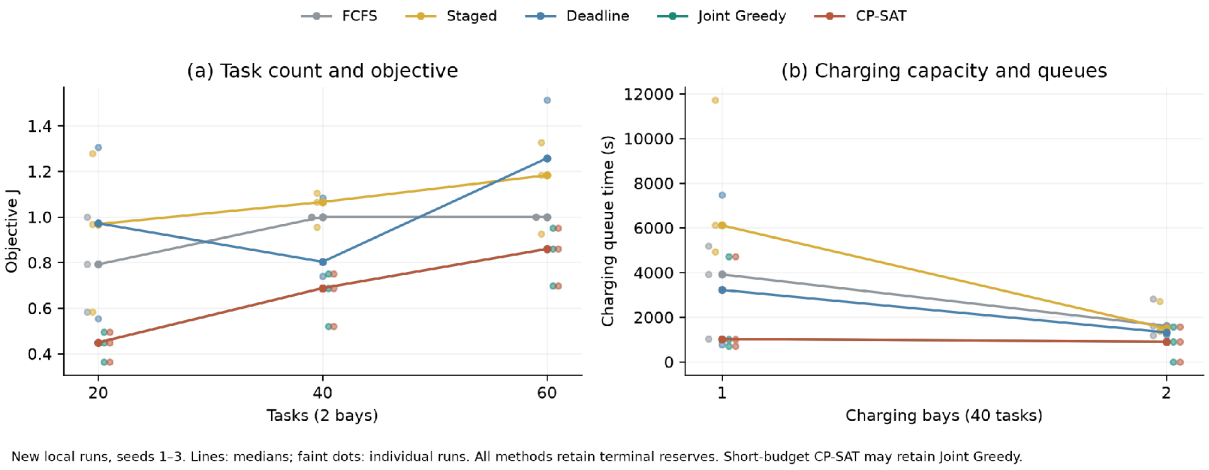


Figure 7.3. Local workload and charging sensitivity. Lines show medians over seeds 1–3 and faint dots individual runs. Left: task count with two bays; right: bay count with 40 tasks.

ber of tasks and their deadlines and priorities together, so they do not isolate a pure priority effect. The four-robot fleet and single intersection stay fixed, so task-count results are not evidence of scalability to larger fleets.

7.5 Online Replanning

On the local 12-task input, rolling CP-SAT performs 18 replans with no incumbent fallbacks. Total runtime is 5.40 s, of which solver wall time accounts for 5.10 s; makespan is 1 237.6 s and J is 0.505. It finishes later than offline CP-SAT but receives tasks only after their release. The archived base mean J of 0.636 also lies between FCFS and offline CP-SAT. These comparisons describe operation with restricted information; they do not separate algorithm quality from future knowledge, and aggregate runtime alone does not establish a worst-case or 95th-percentile replanning latency.

8 Discussion

8.1 Interpretation of Results

The archived and local experiments support the benefit of jointly scheduling task assignment, intersection reservations and charging decisions in the tested system. Joint Greedy reduces the mean composite objective by 20.7 % relative to FCFS and provides a feasible schedule through coordinated resource and battery checks. Offline CP-SAT achieves a further improvement under the archived search budget, although the gain cannot be attributed solely to the solver because CP-SAT also allows flexible charging amounts.

The worked trace identifies a concrete bottleneck: charging to a fixed 95 % target can occupy a bay long enough to delay other robots. Partial charging and task reordering can reduce such delays, but their individual contributions have not been isolated.

The short-budget results also show the importance of computational limits: CP-SAT may return the validated Joint Greedy schedule without improving it. Joint Greedy therefore provides a practical initial solution and fallback, while CP-SAT can improve schedule quality when sufficient search time is available. Results should be read from the measured objective, resource queues and computation time rather than from the solver label alone.

8.2 Limitations

The results apply to the four-robot, fixed-route system and the synthetic workloads considered here. Validation establishes compliance with the modelled resource and battery constraints, but does not prove general

collision avoidance, which requires motion-level conflict models such as that of Li, Fan and Jia [2], or physical deployment safety.

The heuristics use a fixed charging target whereas CP-SAT permits flexible amounts, so the effects of charging flexibility and task reordering cannot be separated. Offline methods also know future tasks, unlike rolling CP-SAT, which prevents a direct comparison under identical information. Battery behaviour is deterministic and linear; energy uncertainty would require stochastic models such as the chance-constrained battery management developed by Ricard et al. [14] for electric buses.

Finally, the archived and local experiments differ in hardware, code revision and solver budget. The archived study uses ten seeds per configuration and the local check uses three, and no formal significance test has been conducted. More controlled comparisons would be needed to isolate the effects of individual scheduling decisions.

9 Conclusion

This project implements joint task, intersection and charging scheduling for a fixed dual-loop system. In the archived base study, reserve-preserving CP-SAT reduces mean J by 53.2 % and makespan by 28.3 % relative to FCFS. The local case links a shorter schedule to reduced bay occupancy and queueing, while the short-budget batch check finds no improvement beyond Joint Greedy, so the solver budget matters alongside the scheduling model.

The terminal reserve trades finite-batch performance for post-task charging access. Rolling planning handles released tasks with frozen commitments, but a fair online ranking still needs baselines with the same information. The present evidence supports the tested synthetic system and its resource rules; broader traffic safety, real-time guarantees and factory deployment require further modelling and measurement.

References

[1] Athira K. A., Divya Udayan J., and U. Subramaniam, "A systematic literature review on multi-robot task allocation," *ACM Computing Surveys*, vol. 57, no. 3, Art. no. 68, 2025, doi:10.1145/3700591.

[2] S. Li, L. Fan, and S. Jia, "A hierarchical solution framework for dynamic and conflict-free AGV scheduling in an automated container terminal," *Transportation Research Part C: Emerging Technologies*, vol. 165, Art. no. 104724, 2024, doi:10.1016/j.trc.2024.104724.

[3] Y. Li, B. Ren, X. Wen, and S.-H. Chung, "An adaptive large neighborhood search method for the AGV scheduling problem with a limited number of chargers," *International Journal of Production Research*, vol. 63, no. 15, pp. 5749–5781, 2025, doi:10.1080/00207543.2025.2462670.

[4] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *Journal of Artificial Intelligence Research*, vol. 31, pp. 591–656, 2008, doi:10.1613/jair.2502.

[5] Y. Li, X. Wen, S. Zhou, and S.-H. Chung, "Smart automated guided vehicle scheduling with flexible battery management: A new formulation and an exact approach," *Computers & Operations Research*, vol. 183, Art. no. 107156, 2025, doi:10.1016/j.cor.2025.107156.

[6] X. Huo, X. He, Z. Xiong, and X. Wu, "Multi-objective optimization for scheduling multi-load automated guided vehicles with consideration of energy consumption," *Transportation Research Part C: Emerging Technologies*, vol. 161, Art. no. 104548, 2024, doi:10.1016/j.trc.2024.104548.

[7] X. Chang, X. Jia, and H. Hu, "Energy-efficient and self-adaptive AGV scheduling approach based on hierarchical reinforcement learning for flexible shop floor," *Computers & Industrial Engineering*, vol. 205, Art. no. 111140, 2025, doi:10.1016/j.cie.2025.111140.

[8] Y. Yao, Q. Liu, L. Fu, X. Li, Y. Yu, L. Gao, and W. Zhou, "A novel mathematical model for the flexible job-shop scheduling problem with limited automated guided vehicles," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 7449–7462, 2025, doi:10.1109/TASE.2024.3356255.

[9] Y. Yao, Q. Liu, X. Li, and L. Gao, "Constraint programming for AGV and machine integrated scheduling problem in flexible manufacturing system," *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 2378–2390, 2026, doi:10.1109/TASE.2025.3650678.

[10] L. Perron and F. Didier, *CP-SAT*, Google OR-Tools, version 9.15.6755, 2026. [Online]. Available: `https://developers.google.com/optimization/cp/cp_solver/`

[11] L. Perron, F. Didier, and S. Gay, "The CP-SAT-LP solver," in *Proc. 29th Int. Conf. Principles and Practice of Constraint Programming (CP 2023)*, LIPIcs, vol. 280, pp. 3:1–3:2, 2023, doi:10.4230/LIPIcs.CP.2023.3.

[12] J. Xin, Q. Yuan, A. D'Ariano, G. Guo, Y. Liu, and Y. Zhou, "Dynamic unbalanced task allocation of warehouse AGVs using integrated adaptive large neighborhood search and Kuhn–Munkres algorithm," *Computers & Industrial Engineering*, vol. 195, Art. no. 110410, 2024, doi:10.1016/j.cie.2024.110410.

[13] A. Berndt, N. van Duijkeren, L. Palmieri, A. Kleiner, and T. Keviczky, "Receding horizon re-ordering of multi-agent execution schedules," *IEEE Transactions on Robotics*, vol. 40, pp. 1356–1372, 2024, doi:10.1109/TRO.2023.3344051.

[14] L. Ricard, G. Desaulniers, A. Lodi, and L.-M. Rousseau, "Chance-constrained battery management for electric bus scheduling," *European Journal of Operational Research*, vol. 336, no. 3, pp. 940–952, 2027, doi:10.1016/j.ejor.2026.05.046.

A Full Model

The implementation uses $V_k = \{0\} \cup I_k \cup Q_k$, where I_k is the set of tasks on robot k's loop and Q_k its optional charging slots. Presence is $z_{ki} = x_{ik}$ for a task, $z_{kq} = y_{kq}$ for a charging visit and $z_{k0} = 1$; assignment x, arc a, activation y and bay b are Boolean. The equations below use seconds and SOC percentage points; internally, time variables are integer ticks in $[0, \bar{H}]$ and node SOC and charging targets lie in $[US_{\min}, 100U]$. The dummy state fixes the initial availability, position and SOC.

$$\sum_{k:\, i \in I_k} x_{ik} = 1, \qquad a_{knn} = 1 - z_{kn} \quad (n \neq 0), \qquad \textsc{Circuit}(a_k). \tag{A.1}$$

Each selected non-self arc belongs to the single circuit through 0, and the dummy self-loop permits an idle robot. A selected arc $a \to n$ with $n \neq 0$ enforces $s_{kn} \geq f_{ka}$ and replays the fixed incoming travel to the pickup or to H. Task copies share s_i, c_i when selected, with $s_i \geq r_i$; loading, loaded travel and unloading advance time by equality. Absent visits have zero times, and inactive crossings and charging waits are zero. Charging visits are used in order, $y_{k,q+1} \leq y_{kq}$ and $s_{k,q+1} \geq f_{kq}$ when $y_{k,q+1} = 1$. Final charge-to-dummy arcs are forbidden, and closing task-to-dummy arcs add no return travel.

$$e = \text{arrival} + w, \qquad l = e + \tau^X_{kr}, \qquad \textsc{NoOverlap}\{[e,\, l + \delta)\}, \tag{A.2}$$

$$\sum_p b_{kqp} = y_{kq}, \qquad u = h + w^C + \tau_{Hp}, \qquad f = u + D + \tau_{pH}, \tag{A.3}$$

$$S^- = S^H - \rho_w w^C - g_{Hp} \geq S_{\min}, \qquad S^+ = S^- + \rho_c D \leq 100, \qquad S^+ - g_{pH} \geq S_{\min}. \tag{A.4}$$

Here τ^X_{kr} is the traversal time of the edge entering X on that crossing. Resource queueing occurs at WA, WB or H, never inside X. The charging equalities apply only to the selected bay, whose optional interval shares that bay's `NoOverlap`. In ticks, an active D lies between 1 and $\lceil 100U/\alpha \rceil$, with $\alpha = \lfloor \rho_c U/T \rfloor$. On every selected path without charging, $S_{\text{out}} = S_{\text{in}} - \sum g - \rho_w \sum w$, including handling. Frozen reservations are mandatory. A last delivery keeps the travel-to-bay consumption plus $3(95 - S_{\min})\,\rho_w/\rho_c$ and a 0.5-point margin above the floor; decoding also checks the actual reachability.

$$\min\ J = \frac{1}{4}\left(\frac{C_{\max}}{\hat{R}_C} + \frac{\sum_i p_i T_i}{\hat{R}_T} + \frac{W_X + W_C}{\hat{R}_W} + \frac{E}{\hat{R}_E} \right), \qquad C_{\max} = \max_i c_i. \tag{A.5}$$

The normalisation denominators are the FCFS terms floored at $(1, 60, 60, 1)$; E excludes replenishment. Rolling solves keep fixed past contributions and normalise on known tasks only. With $T = 1000$ ticks/s

and $U = 60\,000\,000$ SOC units per point, durations, releases, consumption and drain round up, while due dates, initial SOC and charging gain round down. Paths that are not on the grid add 20 ticks of drain per crossing or charging approach as a model guard; this is included in the modelled E but absent from decoded actions. The visit-time bound is $\bar{H} = \lceil T\,[\,t_0 + (|I|+8)(20\,\tau_{\max} + 5000)\,]\,]$, with t_0 the latest initial or release time and $\tau_{\max}$ the longest edge travel time. Robot k receives $\max(q_k^{\text{hint}}, 1, \lfloor |I_k|/5 \rfloor)$ charging slots, where q_k^{hint} is the number of charging visits of robot k in the Joint Greedy hint; an explicit cap also limits the fallback. The valid decoded candidate or the admissible Joint Greedy plan with the lower actual J is returned; if neither is available the run fails. OPTIMAL refers to this bounded integer model.

B Experimental Data and Scheduling Output

B.1 Input Task List

Table B.1 gives all 12 tasks of the worked experiment. The tasks are synthetic and fixed across FCFS, CP-SAT and rolling optimisation; all times are in seconds. R1–R4 start with SOC 50.1, 53.9, 52.2 and 80.8 %; the safety floor is 25 % and two charging bays are enabled.

Table B.1. Complete input task list.

Task	Pickup → delivery	Release	Due	Load	Unload	Priority
T005	B4 → B3	30	835	10	20	2
T012	A4 → A2	40	547	20	15	1
T004	B1 → B2	100	989	10	15	3
T002	B4 → B3	110	729	15	10	3
T008	B2 → B3	130	911	15	15	1
T003	B4 → B3	170	748	20	10	2
T007	B2 → B3	180	1072	20	20	1
T009	A2 → A1	190	829	20	15	1
T010	A2 → A1	190	937	10	20	1
T011	A2 → A1	200	1023	15	20	2
T001	B1 → B2	240	720	15	10	1
T006	A1 → A3	280	944	10	20	3

B.2 Robot Schedules and Charging Records

Table B.2 gives the decoded CP-SAT task order of every robot. “Last delivery” is the completion of the robot’s last transport task; final SOC and consumption refer to its complete action trace. Loop A finishes at 1 165.2 s and Loop B at 947.4 s, so the makespan is 1 165.2 s, with $J = 0.4472$, minimum SOC 25.9 % and no constraint violations.

Table B.2. Robot task sequences and outcomes (CP-SAT).

Robot	Task order	Last delivery (s)	Final SOC (%)	Consumption (pp)
R1	T009, T006	984.8	42.5	33.8
R2	T012, T011, T010	1165.2	46.5	52.0
R3	T005, T004, T007	947.4	42.5	45.4
R4	T001, T008, T002, T003	602.0	51.6	29.2

Table B.3 expands R1’s charging visit into arrival and departure, queueing, charging duration and SOC. FCFS leaves R1 queueing at H for 634.4 s, whereas CP-SAT schedules its arrival after the bay becomes available and charges only to 52.1 %. Travel from H to C1 takes 20 s.

The full outputs of the three strategies contain 36 task records, 372 action records and 265 waypoint visits. Action records keep robot and task identifiers, nodes, start and end times, energy, SOC and resource;

Table B.3. R1 charging visit records.

Policy	Node	Arrival / departure (s)	Queue (s)	Charge (s)	SOC in / out (%)
FCFS	H	116.0 / 750.4	634.4	0	39.5 / 37.4
FCFS	C1	770.4 / 1486.2	0	715.8	35.4 / 95.0
CP-SAT	H	534.4 / 534.4	0	0	27.9 / 27.9
CP-SAT	C1	554.4 / 868.8	0	314.4	25.9 / 52.1

positive energy denotes consumption and negative energy replenishment. The 60-run local supplement and the 700 distinct historical strategy runs are kept separate because their code versions and budgets differ.

C Key Code

Intersection reservation. `JointModel.path` in `optimizer.py` turns a selected crossing into an optional interval of length $\tau^{X}_{kr}+\delta$ (`dt + self.gap`); absent visits have zero entry and wait. One `NoOverlap` per resource then enforces the capacity of X and of each bay.

```
wait, entry = self.var('w_cross'), self.var('e_kr')
self.model.add(entry == at + wait).only_enforce_if(use)
self.model.add(entry == 0).only_enforce_if(~use)
self.model.add(wait == 0).only_enforce_if(~use)
self.resources['X'].append(self.model.new_optional_fixed_size_interval_var(
    entry, dt + self.gap, use, 'X'))
...
for intervals in self.resources.values():   # X, C1, C2
    m.add_no_overlap(intervals)
```

Decoding and safe fallback. A CP-SAT solution is replayed through the planner, validated, given the terminal reserve when enabled, and compared with the Joint Greedy incumbent on the actual objective; otherwise the incumbent is kept and the reason recorded.

```
decoded = joint.decode(solver)              # replay through Planner
build_result(decoded, base.results + tasks, 'cpsat',
             started, opts['norm'])         # independent validation
if opts['terminal_reserve']:                # robots active in this solve
    for k in active:                        # (index set abridged)
        decoded.ensure_reserve(k)           # charger reachable at end
if (selected is None or score(decoded, opts['norm'])
        <= score(selected, opts['norm']) + 1e-7):
    selected = decoded
else:
    metadata['fallback'] = 'joint hint has a lower decoded objective'
```

Reproduction. The repository (snapshot `9bf0b70`) needs Python 3.12 or later and OR-Tools 9.15.6755. The commands below run the tests, rerun the experiment design with the current code (the archived results combine earlier revisions, Section 6) and start the replay:

```
python -m unittest discover -s tests          # 69 tests
python experiments.py --seeds 10 --time-limit 120 --out results
python app.py                                 # browser replay
```